

**Project 1:
Supervised Learning**
alfar24@ru.is



T-504-ITMAL Machine Learning

Contents

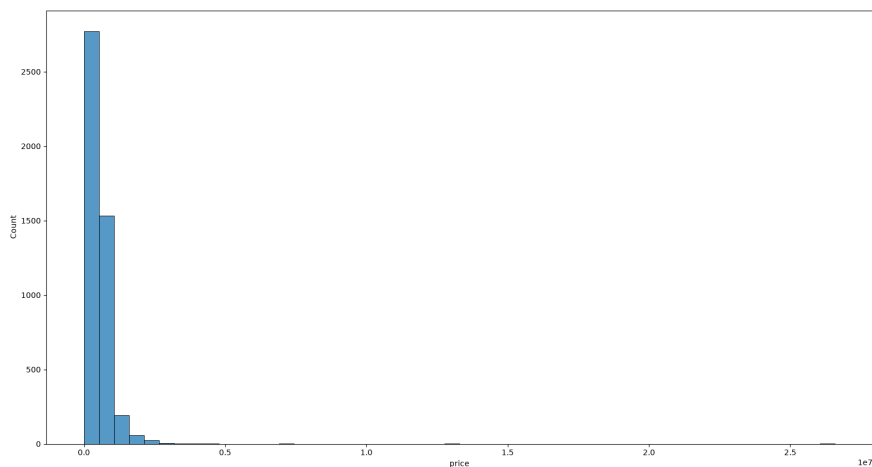
1	Introduction	2
2	Process	5
3	Results	6
4	Conclusion	6
5	Future Work	6
6	Sources	6

1 Introduction

The dataset that I chose is the House Price Prediction dataset. I want to create a model that can effectively predict the price of a house based on its attributes. The dataset I chose gives us a plethora of features to work with. There's a total of 4,600 houses in the data set, each holding the square footage, number of bathrooms, number of bedrooms, how many floors, the condition it is in, the date it was sold, whether or not it has a waterfront and the rating of the view. The benchmark I'm going to be using is a normal Linear Regression, which is a common baseline for datasets like this.

A linear regression requires a dataset of only numerical values; the dataset that I chose has some string values in it, which can't be used in a linear regression. The problematic values here are "date", "street", "city" and "statezip". We can thankfully just drop the street attribute, as every single one of the values in it is unique. We can split the date into 2 attributes (day and month); the year is intentionally dropped here, as the entire data range spans less than a year, so it would actively hurt some models. For the "city" attribute, we can use a "one-hot" method on it, which turns each category into a boolean value. The only drawback here is that it shoots the dimensionality of the dataset up by quite a bit, from 18 to 60. The most

problematic of the string values is the “statezip” value. We can use a one-hot method on this as well, but this attribute shoots the dimensionality up even higher than the city attribute, from 60 to 136. Which is an increase of 76 features, vs. the increase of 42 with the city feature. The resulting Linear Regression gave us an R^2 score of ≈ 0.30 with the training data, and an R^2 score of ≈ 0.60 with the testing data. I also calculated the standard-deviations of the price to be \$427,379.5 for the training data, and \$927,053.6 for the testing data. This is also an extraordinary range, and it clearly hints that the dataset has a few outliers. I made a graph for the prices so I could see if this was the case:



all the houses with a price of \$0. There's a total of 49 properties with a listed price of \$0; this heavily skews the standard-deviation and hurts the model performance. For the zero-price values, we can simply remove these rows, since \$0 does not reflect the actual value of the property (a low bid, or perhaps the house was given away). For the extreme values, we can take the logarithm of them, which compresses the right-skewed tail and reduces the influence of the outliers. The resulting R^2 score of the regression model is 0.805 for the training data, and 0.774 for the testing data. I also calculated an RMSE of 0.269 and a MAE of 0.165 in the logarithmic scale. For the actual Dollar value, I got a MAE of \$110,188.80 and an RMSE of \$486,176.91. This may look like a significant difference at first, but it makes sense: RMSE penalizes larger errors more heavily than MAE does, and the largest values in our dataset are $> 10\times$ the average. We can't do much about this gap without removing the extreme values, which I can't really justify, as they do actually represent real house prices, unlike the zero-price values. Considering everything, these will be our benchmarks.

2 Process

I found an excellent research paper written in the Journal of Research Innovation and Technology [1]. It specifically talks about predicting house prices with machine learning, which is exactly the problem I'm trying to solve. In their research results, the Extra Tree algorithm performed best each time, with Random Forest and Gradient Boosting usually landing in second and third respectively. The best option here would probably be the Extra Tree algorithm, but since we haven't covered that algorithm in the course, I went with the Random Forest algorithm and the Gradient Boost algorithm. The accuracy of the Random Forest algorithm never performed much worse than the extra tree one (97% vs. 99%), so I think it should suffice. I decided to go with a 80/20 split on the data, 80% for the training, and 20% for the testing. The dataset I have is rather small, so I need to concentrate most of it to train my model. When I was first setting up my linear regression, I saw that the R^2 fluctuated wildly based on the random state I used, from 0.6 - 0.06

3 Results

4 Conclusion

5 Future Work

6 Sources

References

- [1] Ogundeji, G. A., Pitts, D. D. A., Sun, Y., & Ghafoor, M. (2024). *Predicting UK Housing Price using Machine Learning Algorithms*. J. Res. Innov. Technol., 3(1), 67-85. [https://doi.org/10.57017/jorit.v3.1\(5\).05](https://doi.org/10.57017/jorit.v3.1(5).05)